TRAVELOOP

Backend Technical Requirements Document

Personalized Travel Planning Platform

Version	1.0
Date	May 2026
Team	Hackathon — Traveloop
Backend	Samarth & Jatin
Frontend	Het & Divya
Stack	Node.js + PostgreSQL

1. Introduction

1.1 Purpose

This Technical Requirements Document (TRD) defines the complete backend architecture, database schema, REST API specifications, authentication strategy, validation rules, and error handling conventions for the Traveloop platform. It serves as the authoritative reference for Samarth and Jatin during the hackathon build, and as the API contract for Het and Divya on the frontend.

1.2 Project Overview

Traveloop is a personalized multi-city travel planning application. Users create trips, add city stops, assign activities, track budgets, maintain packing checklists, write travel notes, and share itineraries publicly. The backend is responsible for all data persistence, business logic, authentication, and exposing a JSON REST API.

1.3 Scope

- User authentication and session management (JWT)
- CRUD operations for trips, stops, cities, activities, notes, packing items
- Budget calculation logic (auto-computed from activities and cost indices)
- Public sharing via shareable tokens (no auth required)
- Seed data for cities and activities
- Admin analytics endpoints (optional Phase 6)

1.4 Out of Scope

- Real-time collaboration or WebSockets
- Payment processing or booking integrations
- Email delivery (forgot password flow mocked in hackathon)
- File upload CDN (photo URLs stored as plain text for hackathon)

2. Technology Stack

Layer	Technology	Notes
Runtime	Node.js 20 LTS	Express.js framework
Language	JavaScript / TypeScript	TypeScript recommended for type safety
Database	PostgreSQL 15	Primary relational store
ORM / Query Builder	Prisma or pg (node-postgres)	Prisma preferred for schema migrations
Authentication	JWT (jsonwebtoken)	Access tokens; 7-day expiry
Password Hashing	bcryptjs	Salt rounds: 12
Validation	Zod	Request body & query param validation
Environment	dotenv	.env file for secrets
API Style	REST JSON	Versioned under /api/v1/
Dev Tools	Nodemon, ESLint, Prettier	Hot reload during development

3. Database Schema

All tables use UUID primary keys generated by gen_random_uuid() (pgcrypto extension). Foreign keys enforce referential integrity with ON DELETE CASCADE where appropriate. Timestamps use TIMESTAMPTZ (timezone-aware).

NOTE Run CREATE EXTENSION IF NOT EXISTS "pgcrypto"; before migrations to enable UUID generation. All migrations should be idempotent using IF NOT EXISTS guards.

3.1 users

Stores all registered users. The is_admin flag gates the optional analytics dashboard.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique user identifier
name	VARCHAR(100)	NOT NULL	Display name
email	VARCHAR(255)	NOT NULL, UNIQUE	Login email address
password_hash	TEXT	NOT NULL	bcrypt hash of password
photo_url	TEXT	NULLABLE	Profile picture URL (optional)
language	VARCHAR(10)	DEFAULT 'en'	Preferred UI language code
is_admin	BOOLEAN	DEFAULT false	Admin access flag
saved_destinations	TEXT[]	DEFAULT '{}'	Array of saved city IDs (denormalized for speed)
created_at	TIMESTAMPTZ	DEFAULT NOW()	Registration timestamp
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last profile update

3.2 trips

Core entity. Each trip belongs to one user and can have many stops. share_token enables public viewing without authentication.

Column	Type	Constraints	Description
id	UUID	PK	Unique trip identifier
user_id	UUID	FK → users.id, NOT NULL	Owning user
name	VARCHAR(150)	NOT NULL	Trip title (e.g. 'Europe Summer 2026')

Column	Type	Constraints	Description
description	TEXT	NULLABLE	Optional trip notes
start_date	DATE	NOT NULL	First day of travel
end_date	DATE	NOT NULL, CHECK > start_date	Last day of travel
cover_photo_url	TEXT	NULLABLE	Hero image URL
is_public	BOOLEAN	DEFAULT false	Whether itinerary is publicly shareable
share_token	UUID	UNIQUE, DEFAULT gen_random_uuid()	Token for public share URL
total_budget	NUMERIC(12,2)	NULLABLE	User-set budget ceiling
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation time
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last modification time

3.3 cities

Reference table seeded by the backend team. Not user-editable. cost_index is a daily average cost estimate in USD, used for automatic budget calculations.

Column	Type	Constraints	Description
id	UUID	PK	Unique city identifier
name	VARCHAR(100)	NOT NULL	City name (e.g. 'Paris')
country	VARCHAR(100)	NOT NULL	Country name
region	VARCHAR(100)	NULLABLE	Geographic region (e.g. 'Western Europe')
cost_index	NUMERIC(8,2)	NOT NULL	Estimated daily cost in USD
popularity_score	INTEGER	DEFAULT 0	Score 0–100 for recommendation ranking
description	TEXT	NULLABLE	Short city description for UI cards
image_url	TEXT	NULLABLE	Representative city photo URL

3.4 trip_stops

Each stop is one city within a trip. order_index (1-based) allows drag-and-drop reordering. The backend must enforce that arrival_date >= trip.start_date and departure_date <= trip.end_date.

Column	Type	Constraints	Description
id	UUID	PK	Unique stop identifier
trip_id	UUID	FK → trips.id ON DELETE CASCADE	Parent trip
city_id	UUID	FK → cities.id	City being visited
arrival_date	DATE	NOT NULL	Arrival date at this city
departure_date	DATE	NOT NULL, CHECK > arrival_date	Departure date from this city
order_index	INTEGER	NOT NULL	Display order within the trip (1, 2, 3...)

3.5 activities

Seeded reference data representing things to do in each city. type values: sightseeing, food, adventure, culture, shopping, nightlife, day_trip.

Column	Type	Constraints	Description
id	UUID	PK	Unique activity identifier
city_id	UUID	FK → cities.id	City this activity belongs to
name	VARCHAR(150)	NOT NULL	Activity name (e.g. 'Eiffel Tower Visit')
type	VARCHAR(50)	NOT NULL	Category: sightseeing, food, adventure...
cost	NUMERIC(8,2)	NOT NULL, DEFAULT 0	Default cost per person in USD
duration_mins	INTEGER	NULLABLE	Estimated duration in minutes
description	TEXT	NULLABLE	Short activity description
image_url	TEXT	NULLABLE	Activity photo URL

3.6 stop_activities

Junction table linking a trip stop to its selected activities. custom_cost overrides the default activity cost when the user adjusts it. scheduled_time records what time the activity is planned.

Column	Type	Constraints	Description
id	UUID	PK	Unique record identifier
stop_id	UUID	FK → trip_stops.id ON DELETE CASCADE	Parent stop
activity_id	UUID	FK → activities.id	Selected activity
scheduled_time	TIME	NULLABLE	Planned time (e.g. 09:00)
custom_cost	NUMERIC(8,2)	NULLABLE	Override cost; falls back to activities.cost if NULL
notes	TEXT	NULLABLE	User notes for this activity slot

3.7 packing_items

Per-trip packing checklist items. category values: clothing, documents, electronics, toiletries, gear, other.

Column	Type	Constraints	Description
id	UUID	PK	Unique item identifier
trip_id	UUID	FK → trips.id ON DELETE CASCADE	Parent trip
name	VARCHAR(150)	NOT NULL	Item name (e.g. 'Passport')
category	VARCHAR(50)	NOT NULL, DEFAULT 'other'	Item category for grouping
is_packed	BOOLEAN	DEFAULT false	Whether item has been packed
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation time

3.8 trip_notes

Free-text notes scoped to a trip or optionally to a specific stop. If stop_id is NULL the note is trip-level. Sorted by created_at descending in API responses.

Column	Type	Constraints	Description
id	UUID	PK	Unique note identifier
trip_id	UUID	FK → trips.id ON DELETE CASCADE	Parent trip
stop_id	UUID	FK → trip_stops.id NULLABLE	Optional: scope to a specific stop
content	TEXT	NOT NULL	Note body text
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last edit timestamp

3.9 Relationship Summary

CASCADE	Deleting a user cascades to trips. Deleting a trip cascades to trip_stops, packing_items, and trip_notes. Deleting a stop cascades to stop_activities and stop-scoped notes.

4. Authentication

4.1 Strategy

Traveloop uses stateless JWT (JSON Web Token) authentication. On successful login or registration, the server issues a signed access token. The frontend stores this token (localStorage or memory) and sends it in the Authorization header of every protected request.

4.2 Token Specification

- Algorithm: HS256
- Expiry: 7 days (604800 seconds)
- Payload: { sub: userId, email, is_admin, iat, exp }
- Secret: stored in JWT_SECRET environment variable (min 32 chars)

4.3 Request Header Format

Header	Authorization: Bearer <token>

4.4 Auth Middleware

All protected routes pass through an authenticate middleware that:

- Extracts the Bearer token from the Authorization header
- Verifies the signature using JWT_SECRET
- Rejects expired or malformed tokens with 401 Unauthorized
- Attaches the decoded payload to req.user for downstream handlers

4.5 Password Policy

- Minimum 8 characters
- Hashed with bcryptjs, salt rounds: 12
- Plaintext password is never logged or stored

5. API Endpoints Reference

All endpoints are prefixed with /api/v1. JSON is the only supported content type. All protected endpoints require the Authorization: Bearer <token> header. Timestamps in responses are ISO 8601 UTC strings.

5.1 Authentication

Method	Endpoint	Auth	Description
POST	/api/v1/auth/register	Public	Create a new user account. Returns JWT token.
POST	/api/v1/auth/login	Public	Authenticate with email & password. Returns JWT token.
GET	/api/v1/auth/me	JWT	Return the currently authenticated user's profile.

POST /auth/register — Request Body

Body	{ name: string (required), email: string (required, valid email), password: string (required, min 8 chars) }

POST /auth/login — Request Body

Body	{ email: string (required), password: string (required) }

Auth Response Schema

Response	{ token: string, user: { id, name, email, photo_url, language, is_admin, created_at } }

5.2 Users

Method	Endpoint	Auth	Description
GET	/api/v1/users/me	JWT	Get current user profile including saved_destinations.
PATCH	/api/v1/users/me	JWT	Update name, photo_url, language, or saved_destinations.

Method	Endpoint	Auth	Description
DELETE	/api/v1/users/me	JWT	Delete account. Cascades to all trips and related data.
PATCH	/api/v1/users/me/password	JWT	Change password. Requires current_password verification.

5.3 Trips

Method	Endpoint	Auth	Description
GET	/api/v1/trips	JWT	List all trips for authenticated user. Supports ?upcoming=true filter.
POST	/api/v1/trips	JWT	Create a new trip. Returns created trip with share_token.
GET	/api/v1/trips/:id	JWT	Get full trip with stops, activities, budget summary.
PATCH	/api/v1/trips/:id	JWT	Update trip name, dates, description, cover_photo_url, total_budget.
DELETE	/api/v1/trips/:id	JWT	Delete trip and all related data (cascade).
PATCH	/api/v1/trips/:id/share	JWT	Toggle is_public. Returns the share URL with share_token.
GET	/api/v1/public/:token	Public	Fetch read-only itinerary view by share_token. No auth required.
POST	/api/v1/public/:token/copy	JWT	Copy a public trip into the authenticated user's account.
GET	/api/v1/trips/:id/budget	JWT	Return computed cost breakdown by category plus daily averages.

POST /trips — Request Body

Body	{ name: string (required), start_date: date string YYYY-MM-DD (required), end_date: date string YYYY-MM-DD (required, > start_date), description?: string, cover_photo_url?: string, total_budget?: number }

5.4 Trip Stops

Method	Endpoint	Auth	Description
GET	/api/v1/trips/:id/stops	JWT	List all stops for a trip ordered by order_index.
POST	/api/v1/trips/:id/stops	JWT	Add a city stop to a trip. Auto-assigns next order_index.
PATCH	/api/v1/trips/:id/stops/:stopId	JWT	Update a stop's dates.
DELETE	/api/v1/trips/:id/stops/:stopId	JWT	Remove a stop and its activities.
PATCH	/api/v1/trips/:id/stops/reorder	JWT	Reorder stops. Body: { order: [stopId1, stopId2, ...] }.

POST /stops — Request Body

Body	{ city_id: UUID (required), arrival_date: YYYY-MM-DD (required), departure_date: YYYY-MM-DD (required) }

5.5 Cities

Method	Endpoint	Auth	Description
GET	/api/v1/cities	JWT	Search cities. Supports ?q=paris, ?region=Europe, ?sort=popularity.
GET	/api/v1/cities/:id	JWT	Get city details including cost_index and popularity_score.
GET	/api/v1/cities/:id/activities	JWT	List all activities for a city. Supports ?type=&max_cost= filters.

5.6 Stop Activities

Method	Endpoint	Auth	Description
GET	/api/v1/trips/:id/stops/:stopId/activities	JWT	List all activities added to a specific stop.

Method	Endpoint	Auth	Description
POST	/api/v1/trips/:id/stops/:stopId/activities	JWT	Add an activity to a stop. Body: { activity_id, scheduled_time?, custom_cost?, notes? }.
PATCH	/api/v1/trips/:id/stops/:stopId/activities/:saId	JWT	Update scheduled_time, custom_cost, or notes for a stop activity.
DELETE	/api/v1/trips/:id/stops/:stopId/activities/:saId	JWT	Remove an activity from a stop.

5.7 Packing Checklist

Method	Endpoint	Auth	Description
GET	/api/v1/trips/:id/packing	JWT	Get all packing items for a trip, grouped by category.
POST	/api/v1/trips/:id/packing	JWT	Add a packing item. Body: { name, category }.
PATCH	/api/v1/trips/:id/packing/:itemId	JWT	Toggle is_packed or update name/category.
DELETE	/api/v1/trips/:id/packing/:itemId	JWT	Remove a packing item.
DELETE	/api/v1/trips/:id/packing/reset	JWT	Set all items is_packed = false (reset checklist).

5.8 Trip Notes

Method	Endpoint	Auth	Description
GET	/api/v1/trips/:id/notes	JWT	List all notes for a trip sorted by created_at DESC. Filter ?stop_id= for stop-scoped notes.
POST	/api/v1/trips/:id/notes	JWT	Create a note. Body: { content, stop_id? }.
PATCH	/api/v1/trips/:id/notes/:noteId	JWT	Update note content. Updates updated_at automatically.
DELETE	/api/v1/trips/:id/notes/:noteId	JWT	Delete a note.

5.9 Admin Analytics (Optional)

Method	Endpoint	Auth	Description
GET	/api/v1/admin/stats	JWT + Admin	Platform stats: total users, trips, most popular cities.
GET	/api/v1/admin/users	JWT + Admin	Paginated user list with trip counts.
GET	/api/v1/admin/trips	JWT + Admin	All trips with owner info and activity counts.

6. Budget Calculation Logic

The GET /trips/:id/budget endpoint auto-computes a cost breakdown. No separate budget table is needed — all data is derived from existing tables.

6.1 Computation Formula

- Activities cost: SUM of COALESCE(stop_activities.custom_cost, activities.cost) for all stop_activities in the trip
- Accommodation estimate: SUM of (cities.cost_index * 0.45 * nights_at_stop) for all stops — 45% of cost index allocated to stay
- Meals estimate: SUM of (cities.cost_index * 0.30 * nights_at_stop) — 30% to meals
- Transport estimate: (number_of_stops - 1) * 80 — flat $80 per city transition
- Total estimated: activities + accommodation + meals + transport
- Budget remaining: trips.total_budget - total_estimated (NULL if no budget set)

6.2 Response Schema

Response	{ breakdown: { activities, accommodation, meals, transport }, total_estimated, total_budget, remaining, daily_avg, over_budget: boolean, stops: [{ city, nights, cost }] }

7. Validation Rules

All incoming request bodies are validated using Zod schemas before reaching database queries. Invalid requests return 422 Unprocessable Entity with a structured errors array.

7.1 Global Rules

- UUIDs in path parameters validated via regex before DB query
- String fields trimmed of whitespace before storage
- Date strings must parse to valid JS Date objects
- Numeric fields (cost, budget) must be >= 0
- Pagination: page >= 1, limit <= 100, default limit 20

7.2 Entity-Specific Rules

- users.email: RFC 5322 format, max 255 chars, lowercase-normalized
- users.password: min 8 chars, max 72 chars (bcrypt limit)
- trips.end_date: must be strictly after start_date
- trips.name: min 1 char, max 150 chars
- trip_stops.arrival_date: must be within trip start_date / end_date range
- trip_stops.departure_date: must be after arrival_date and within trip end_date
- stop_activities: activity must belong to the stop's city (enforce in service layer, not just DB)
- packing_items.name: min 1 char, max 150 chars
- trip_notes.content: min 1 char, max 5000 chars

8. Error Handling

All errors follow a consistent JSON envelope. The global error middleware catches any unhandled errors and returns appropriate HTTP codes.

8.1 Error Response Envelope

Format { success: false, error: { code: string, message: string, details?: any } }

8.2 HTTP Status Code Reference

HTTP Code	Error Code	Description
200	SUCCESS	Request succeeded
201	CREATED	Resource created successfully
400	BAD_REQUEST	Malformed request (missing required field, invalid JSON)
401	UNAUTHORIZED	Missing or invalid JWT token
403	FORBIDDEN	Authenticated but not permitted (e.g. accessing another user's trip)
404	NOT_FOUND	Resource does not exist or is not accessible to this user
409	CONFLICT	Duplicate resource (e.g. email already registered)
422	VALIDATION_ERROR	Request body failed Zod schema validation
500	INTERNAL_ERROR	Unexpected server error — log and do not expose details

9. Environment Configuration

9.1 Required Environment Variables

Variable	Example Value	Notes
DATABASE_URL	postgresql://user:pass@localhost:5432/traveloop	Full Postgres connection string
JWT_SECRET	your-very-long-secret-key-here-32chars	Min 32 characters for HS256
PORT	3000	HTTP server port
NODE_ENV	development | production	Controls logging and error detail
BCRYPT_SALT_ROUNDS	12	Password hashing cost (default 12)
CORS_ORIGIN	http://localhost:5173	Frontend origin for CORS whitelist

9.2 Project Structure

- src/
  - index.js — Express app entry point, middleware registration
  - routes/ — One file per resource (auth.routes.js, trips.routes.js...)
  - controllers/ — Request/response handling, calls service layer
  - services/ — Business logic (budget calculation, share token, copy trip)
  - middleware/ — authenticate.js, errorHandler.js, validate.js
  - schemas/ — Zod schemas for request validation
  - db/ — Prisma schema or pg pool setup
- prisma/
  - schema.prisma — Prisma data model (if using Prisma)
  - migrations/ — Auto-generated migration files
  - seed.js — City and activity seed script
- .env — Environment variables (git-ignored)
- package.json

9.3 Getting Started

1. Clone the repo and run npm install
2. Copy .env.example to .env and fill in DATABASE_URL and JWT_SECRET
3. Run npx prisma migrate dev --name init to create all tables
4. Run node prisma/seed.js to populate cities and activities data
5. Run npm run dev to start the server with hot reload

10. Seed Data Requirements

The seed script must run before the frontend can function. Samarth and Jatin should prioritize this in Phase 1 so Het and Divya can work against real data from day one.

10.1 Cities to Seed (minimum 20)

City	Country	Region	Cost/Day (USD)	Popularity
Paris	France	Western Europe	$150	95
Tokyo	Japan	East Asia	$120	98
New York	USA	North America	$200	97
Barcelona	Spain	Southern Europe	$110	92
Bangkok	Thailand	Southeast Asia	$55	90
Amsterdam	Netherlands	Western Europe	$130	88
Dubai	UAE	Middle East	$180	91
Rome	Italy	Southern Europe	$115	93
Singapore	Singapore	Southeast Asia	$140	89
London	UK	Western Europe	$180	96
Bali	Indonesia	Southeast Asia	$45	87
Istanbul	Turkey	Middle East	$70	85
Prague	Czech Republic	Eastern Europe	$75	84
Sydney	Australia	Oceania	$160	88
Lisbon	Portugal	Southern Europe	$90	86
Mexico City	Mexico	North America	$65	83
Cairo	Egypt	North Africa	$50	80
Vienna	Austria	Western Europe	$120	85
Seoul	South Korea	East Asia	$95	87
Cape Town	South Africa	Sub-Saharan Africa	$80	82

10.2 Activities per City

Seed at least 5 activities per city across different types (sightseeing, food, adventure, culture). Example for Paris:

- Eiffel Tower Visit — sightseeing — $28 — 120 mins
- Seine River Cruise — sightseeing — $22 — 90 mins
- Louvre Museum — culture — $20 — 180 mins
- Montmartre Food Tour — food — $65 — 150 mins
- Day Trip to Versailles — day_trip — $40 — 360 mins

11. Non-Functional Requirements

11.1 Performance

- All API endpoints must respond within 500ms under local dev load
- City search should use a PostgreSQL index on cities.name (ILIKE queries)
- Add index on trips.user_id and trip_stops.trip_id for frequent JOIN patterns

11.2 Security

- CORS restricted to frontend origin defined in CORS_ORIGIN env var
- All user-owned resources validate req.user.id === resource.user_id before returning data
- SQL injection prevention: use parameterized queries only (Prisma or pg $1/$2 placeholders)
- Secrets must not appear in logs, error responses, or console output

11.3 Hackathon Constraints

- No email delivery required — forgot password flow can return a success message without actually sending
- File uploads are out of scope — photo_url fields accept externally hosted URL strings
- Rate limiting is not required for the hackathon demo
- HTTPS not required for local demo; use HTTP

12. Team Assignment Summary

Phase	Feature	Assigned To	Est. Time
Phase 1	DB migrations + seed data	Samarth & Jatin	1 hr
Phase 1	Auth endpoints (register, login, me)	Jatin	1 hr
Phase 2	Trips CRUD (/trips, /trips/:id)	Samarth	1 hr
Phase 2	Trip sharing (share toggle, public view)	Samarth	45 min
Phase 3	City search + city detail	Jatin	45 min
Phase 3	Activity search + filtering	Samarth	45 min
Phase 3	Trip stops CRUD + reorder	Jatin	1 hr
Phase 3	Stop activities CRUD	Samarth	45 min
Phase 4	Budget calculation endpoint	Jatin	1 hr
Phase 4	Packing checklist CRUD	Samarth	30 min
Phase 5	Trip notes CRUD	Jatin	30 min
Phase 5	User profile update + delete account	Samarth	30 min
Phase 6	Admin analytics (optional)	Both	1 hr

TIP Samarth and Jatin should work in parallel on different endpoints. As soon as an endpoint is ready, share the base URL + sample curl command in your group chat so Het and Divya can integrate immediately.